TASK20: Mini Git Branch App

Aim: To develop a Git-like application that supports branching, merging, rebasing, and conflict resolution using a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Store branches and commits in memory
4. Create default branch (main)
5. Add feature to create new branch
6. Commit changes to selected branch
7. Implement merge logic
8. Detect conflicts (same file modified)
9. Show conflict message
10. Resolve conflicts manually
11. Implement rebase logic
12. Display branch history and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
   <title>Mini Git Branching</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
```

```
    <h2>Mini Git Branching</h2>
    <input id="branchName" placeholder="New Branch Name">
    <button onclick="createBranch()">Create Branch</button>
    <input id="message" placeholder="Commit Message">
    <button onclick="commit()">Commit</button>
    <button onclick="merge()">Merge</button>
    <button onclick="rebase()">Rebase</button>
    <h3>Branches</h3>
    <ul id="branches"></ul>
    <h3>History</h3>
    <ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
    font-family: Arial;
    background: #1e1e2f;
    color: white;
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}
.container {
```

```
    text-align:center;
}
input, button {
    margin: 5px;
    padding: 8px;
}
li {
    background: #333;
    margin: 5px;
    padding: 5px;
}
```

SCRIPT.JS:

```
function createBranch() {
    const name = document.getElementById("branchName").value;
    fetch("/branch", {
        method: "POST",
        headers: {"Content-Type":"application/json"},
        body: JSON.stringify({ name })
    }).then(load);
}
function commit() {
    const message = document.getElementById("message").value;
    fetch("/commit", {
        method: "POST",
        headers: {"Content-Type":"application/json"},
        body: JSON.stringify({ message })
```

```
    }).then(load);
}
function merge() {
    fetch("/merge", { method: "POST" })
    .then(res => res.json())
    .then(alert)
    .then(load);
}
function rebase() {
    fetch("/rebase", { method: "POST" })
    .then(load);
}
function load() {
    fetch("/data")
    .then(res => res.json())
    .then(data => {
        let b = document.getElementById("branches");
        let h = document.getElementById("history");
        b.innerHTML = "";
        h.innerHTML = "";
        data.branches.forEach(br => {
            let li = document.createElement("li");
            li.innerText = br;
            b.appendChild(li);
        });
data.history.forEach(c => {
```

```
            let li = document.createElement("li");
            li.innerText = c;
            h.appendChild(li);
        });
    });
}
load();
```

SERVER.JS:

```
const express = require("express");
const db = require("./db");
const app = express();
app.use(express.json());
app.use(express.static(__dirname));
app.post("/branch", (req, res) => {
    db.createBranch(req.body.name);
    res.json({ msg: "Branch created" });
});
app.post("/commit", (req, res) => {
    db.commit(req.body.message);
    res.json({ msg: "Committed" });
});
app.post("/merge", (req, res) => {
    const result = db.merge();
    res.json({ msg: result });
});
app.post("/rebase", (req, res) => {
```

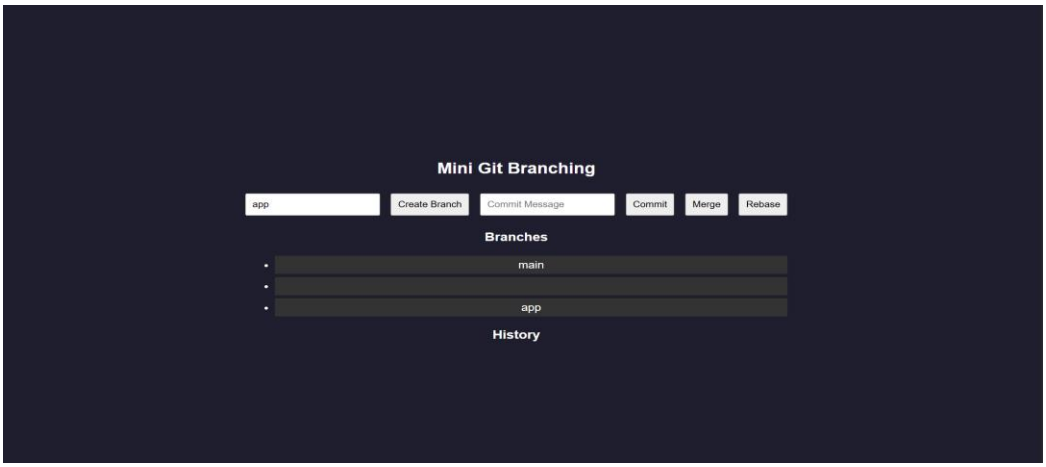
```
    db.rebase();
    res.json({ msg: "Rebased" });
});
app.get("/data", (req, res) => {
    res.json(db.getData());
});
app.listen(3000, () => console.log("http://localhost:3000"));
```

DB.JS:

```
let branches = ["main"];
let current = "main";
let commits = {
    main: []
};
function createBranch(name) {
    branches.push(name);
    commits[name] = [...commits[current]];
}
function commit(msg) {
    commits[current].push(msg);
}
function merge() {
let mainCommits = commits["main"];
    let featureCommits = commits[current];
    if (mainCommits.length && featureCommits.length) {
        return "Merge Conflict Detected!";
    }
```

```
commits["main"] = [...mainCommits, ...featureCommits];
    return "Merged Successfully";
}
function rebase() {
    commits[current] = [...commits["main"], ...commits[current]];
}
function getData() {
    return {
        branches,
        history: commits[current]
    };
}
module.exports = { createBranch, commit, merge, rebase, getData };
```

OUTPUT:



RESULT: A Git-like branching application was successfully developed to simulate branching, merging, rebasing, and conflict resolution through a web interface**.**